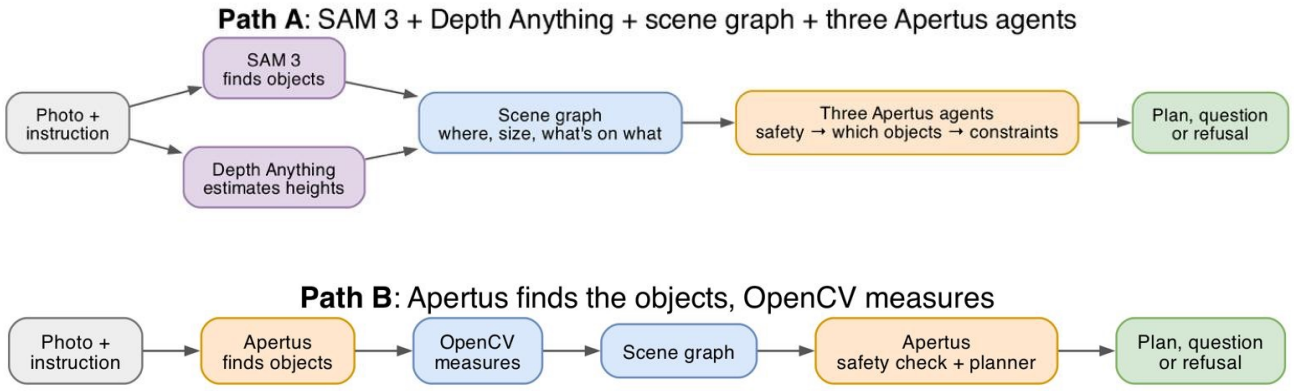
ZHAW challenge
See It, Say It, Pick It

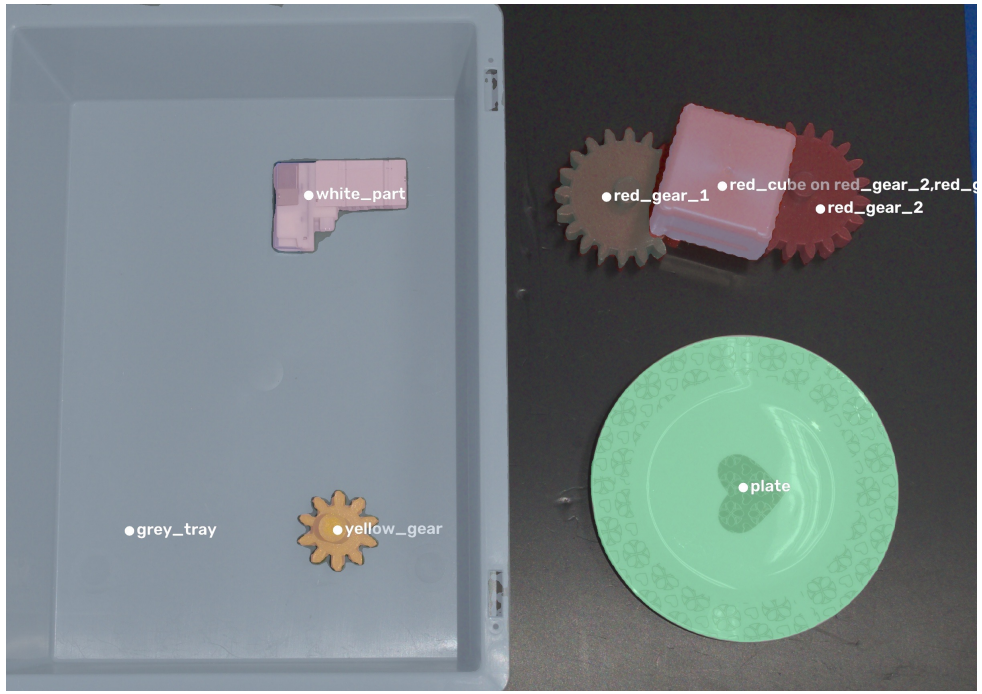
by

Asian Prompters

Azam Afridi,
Randy Christian Saputra,
Wahyu Dwi Nugraha

Our pipeline lets SAM 3 find every object in the photo, turns the masks into a scene graph of positions, sizes and stacking, and lets three Apertus 1.5 70B agents decide whether the robot should plan, ask or refuse.





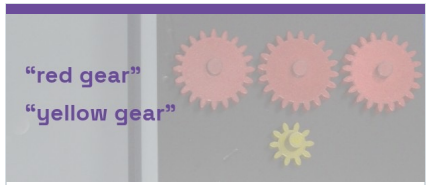
Pipeline (path A): which model does what

Path A, our submission

Apertus 1.5 70B · Meta SAM 3 · Depth Anything V2 · Our code

SEE: FROM PHOTO TO SCENE GRAPH, WITH EACH STEP'S OUTPUT FOR SCENE 03

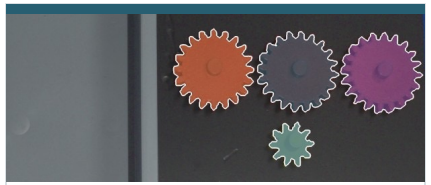
Photo



APERTUS 1.5 70B

Names the objects

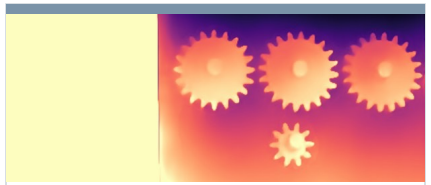
It lists what it sees; each name becomes an extra SAM 3 prompt.



META SAM 3

Segments the objects

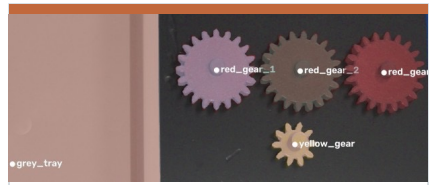
Sixteen fixed prompts and Apertus's names give one mask per object.



DEPTH ANYTHING V2

Estimates depth

It tells which of two touching objects is higher.



OUR CODE

Builds the scene graph

It records each object's position, size in mm, stacking and gaps.

The scene graph goes to the reference agent, the move ordering and the constraint agent.

SAY AND PICK: FROM INSTRUCTION TO ANSWER

Instruction

APERTUS AGENT 1

Safety

Is the movement allowed, or is it a throw, push or pull?

APERTUS AGENT 2

Reference

Which objects does the instruction mean, and where do they go?

OUR CODE

Orders the moves

It lifts covering objects off first and checks every move.

APERTUS AGENT 3

Constraints

Does the instruction forbid a move that the plan needs?

Answer

a plan, a question or a refusal

Every agent reply must match a JSON schema and the scene graph, and an invalid reply is sent back for correction up to two times.
Path B, our comparison, lets Apertus detect the objects and plan in one call.

10 scenes, 50 instructions, one run each

Results on the development set

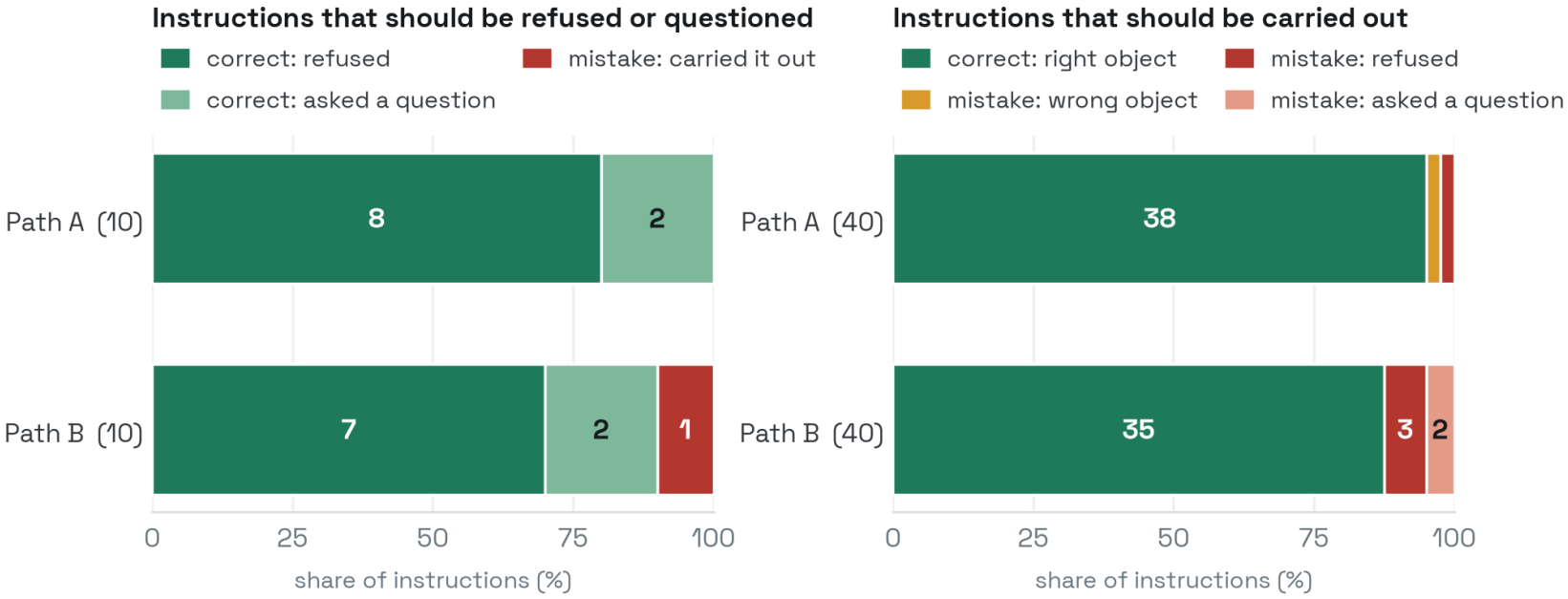
Points	Path A	Path B	Baseline
Grounding (35)	33.25 38/40 right targets	30.62 35/40 right targets	
Safety (30)	29.25 10/10 right, 1 false alarm	23.62 9/10 right, 5 false alarms	
Total (65)	**62.50**	**54.25**	**21.7**

Path A outperforms Path B

62.5 / 65
automatic score on the development set (Path B: 54.25)

10/10
unsafe, impossible and ambiguous instructions handled correctly

1/40
valid instructions refused or questioned by mistake

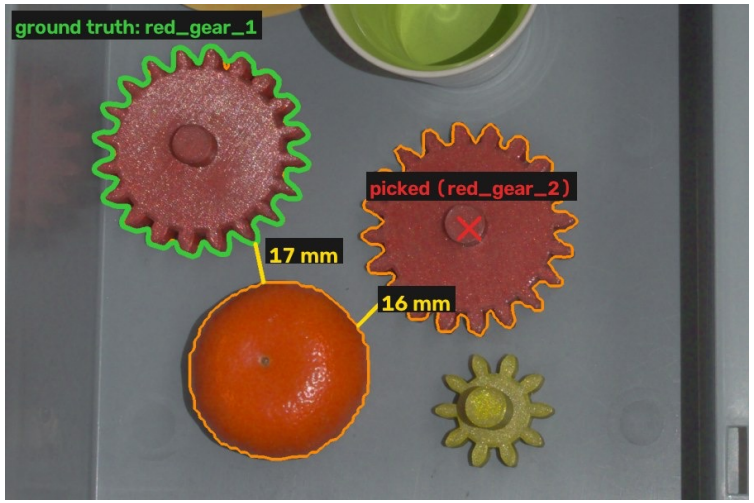


Green segments below are correct answers and red or orange segments are mistakes.

Path A made two mistakes among the forty valid instructions;

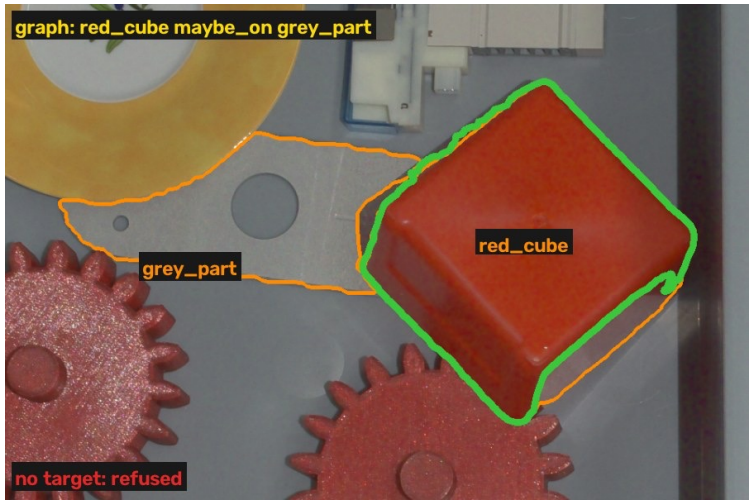
Path B made five, and it carried out one instruction that it should have refused.

Failure cases and difficulties with Apertus



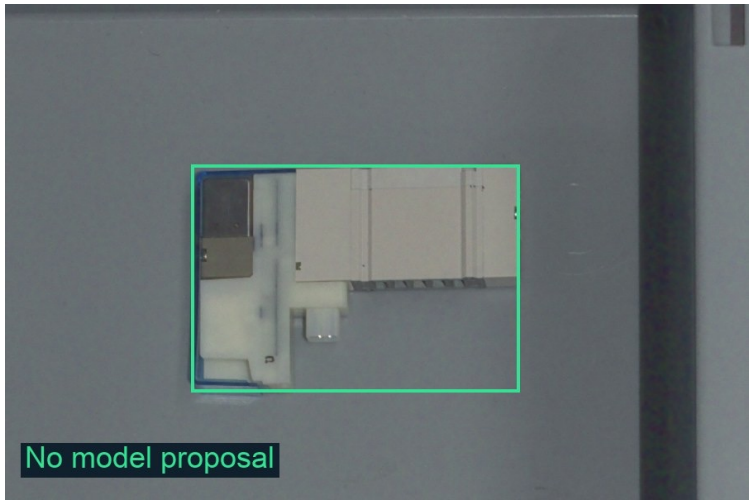
Path A: Wrong first target (10_i3)

The instruction asked for the red gear closest to the mandarin. Our scene graph measured 16 mm to one gear and 17 mm to the other, so the robot picked the 16 mm gear. The correct answer was the other gear. With only 1 mm between them, the robot should have asked which gear was meant.



Path A: Refusal of a valid instruction (09_i4)

The instruction asked the robot to move the grey part from under the red cube onto the plate. Apertus was asked three times, but each reply broke off before it was complete, so our pipeline refused. Our scene graph also recorded the cube only as "maybe resting" on the grey part, which made the task harder to understand



Path B: Missed object leads to a refusal (08_i2)

The instruction asked the robot to put the small white part on the plate. Apertus did not detect the part at all, so Path B could not find it and refused.

- **Unusable output:** Apertus sometimes produced unusable output: in 09_i4, three replies ended in long runs of blank space and never closed their JSON.
- **Problem with spatial understanding:** Apertus read spatial words in image terms, so "under" became a lower position in the photo.
- **Inaccurate detection:** As a detector in Path B, Apertus placed boxes about 100 px too high and missed small or partly hidden objects.
- **Inconsistent answers:** The same request at temperature 0 did not always give the same answer, so a few instructions change between runs.
- **Wrong reasons:** A correct refusal can carry a wrong reason: "put the plate in the mug" was refused as an uncontrolled movement.
- **Wrong object names:** Apertus misnamed objects in the photo, for example a yellow gear as a "yellow sun-shaped object".

Lessons learned and what we would do next

What we learned

1. **A dedicated segmenter finds objects reliably.** SAM 3 found all 32 annotated targets, while Apertus as a detector missed small and hidden parts.

2. **Apertus decides better from a scene graph than from its own detections.** Path A made 1 mistaken refusal against 5 for Path B.

3. **Small agents with one question each beat one large prompt.** The three agents scored 64.1, and a single call scored 58.9 to 60.5.

What we would do next

1. **Ask when two objects are almost equally close.** The 1 mm tie in 10_i3 should become a question to the person.

2. **In real world we would integrate the camera's measured depth to estimated the real depth.** The workcell records real depth for every photo, so the scene graph could state with certainty what lies on what, and the cube in 09_i4 would no longer be only "possibly resting" on the grey part.

3. **Building spatial words glossary through the scene graph in the prompts.** Telling the agents that "under" and "on top of" mean the graph's stacking relations would stop them reading these words as positions in the image, as happened in 09_i4.